

Step 2 — A Real Terminal Chat With Memory and Live Streaming — Hints

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 3 depth levels: **Basic** (the plain idea), **Intermediate** (proper Python), **Advanced** (what happens once the conversation itself gets big). Read Basic first even if you already know Python — it's the fastest way to spot exactly what each deeper level adds.

- [Hint 1 — Memory is a growing list; streaming is just how you print it](#)
- [Hint 2 — The plan, and almost the whole thing](#)

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 1 — Memory is a growing list; streaming is just how you print it

Basic Version

The model itself has no memory between calls — Doc03 calls this "statelessness." So "memory" in your app is really just you: your code keeps a growing list of every message so far, and resends the whole list every single time.

You need to change `send_message()` so it takes that whole list in, and hands back the growing list with the new reply added — not just a single reply string. Then wrap it in a loop that keeps asking the user for input.

Separately, "streaming" just means asking the API to send the reply back piece by piece, so you can print it as it arrives instead of waiting for the whole thing.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Two signatures to build, both taking and returning the growing `messages` list:

```
def send_message(client: OpenAI, messages: list[dict]) -> list[dict]:
```

```
def stream_message(client: OpenAI, messages: list[dict]) -> list[dict]:
```

Each call appends the user's new message to `messages` before calling the API, then appends the assistant's reply to that same list before returning it — so the caller always has the full, up-to-date history, and never has to manage it by hand outside these functions.

For streaming, the API call changes to `stream=True`, which returns an iterator of small chunks instead of one full response object. Look specifically at: `chunk.choices[0].delta.content` — in streaming mode, each chunk carries a small `delta` (a partial change), not a full message; the first and last chunks in particular often have `content=None`, so always guard with `if piece:` before using it. `print(piece, end="", flush=True)` — `end=""` stops Python from adding a newline after every tiny piece, and `flush=True` forces it to appear immediately instead of sitting in a buffer. Keep a `full_reply = ""` and do `full_reply += piece` on every chunk, so you have the complete text to append to `messages` once the stream ends — the stream itself doesn't hand you a "final" complete string automatically.

Sketch the loop in `main.py`: read input, call `stream_message`, print as it streams, repeat — before writing the function bodies.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

There's a problem this step's basic design never solves: the `messages` list only ever grows. Every turn appends two more entries and nothing ever comes back out. That's fine for a short test conversation, but think about what happens in a session that runs for an hour.

Tie this back to Doc03's context-window math. Every model has a maximum number of tokens it can process in one call. That limit covers the *entire* `messages` list you send, plus the reply it generates — not just the newest message. As a rough rule of thumb, one token is about 4 characters of English text, or roughly $\frac{3}{4}$ of a word (see Doc03 for the exact numbers for your model). A long enough conversation, resent in full on every turn the way Step 2 does it, eventually gets close to that limit. Once it crosses the limit, the API call itself fails with a context-length error. Step 3 teaches you to catch that error so the app doesn't crash — but catching the crash is a last line of defense, not a plan. By the time it fires, the user is mid-conversation and the app just stops working for them.

A first-pass trimming strategy heads this off before it becomes a crash. Instead of letting `messages` grow forever, decide on a policy up front: keep a rough running estimate of how many tokens the list would cost, and once that crosses a threshold well under the model's real limit (leave room for the reply itself), start dropping the oldest non-system messages. This is called "first-pass" on purpose — it's crude. It drops old messages by their age or size, not by whether they still matter to the conversation. A smarter version, one that summarizes what got dropped or keeps the more important turns, is something you'll build later once you're managing memory inside a real graph (Project 3 and beyond). For this project, the goal is simpler: don't let the app silently grow its way into a crash.

Difference between Basic, Intermediate, and Advanced: Basic and Intermediate build a `messages` list that grows without limit, and trust the conversation never gets long enough to matter. Advanced treats that as a real failure you can expect, not a rare edge case, and adds a rough, honest strategy — estimate size, trim the oldest entries once you're near the limit — so a long conversation slows down gracefully instead of crashing outright.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 2 — The plan, and almost the whole thing

Basic Version

The plan, in plain steps:

```
start with an empty messages list

loop forever:
    read what the user typed
    add it to messages as a "user" message

    call the model with the whole messages list, streaming on
    print each piece as it arrives
    build the full reply out of all the pieces

    add the full reply to messages as an "assistant" message
    (loop back and read the next thing the user types)
```

Here's almost the whole thing — try running it and reading it line by line:

```
def stream_message(client, messages):
    stream = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=messages,
        stream=True,
    )
    full_reply = ""
    for chunk in stream:
        piece = chunk.choices[0].delta.content
        if piece:
            print(piece, end="", flush=True)
            full_reply += piece
    print()
    messages.append({"role": "assistant", "content": full_reply})
    return messages
```

What's missing: type hints, the non-streaming `send_message()` (same idea, no `stream=True`, just append and return), and the `main.py` loop that reads input and calls this.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

The same plan, fully typed, with `send_message()` alongside it:

```
def send_message(client: OpenAI, messages: list[dict]) -> list[dict]:
    response = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=messages,
    )
    reply = response.choices[0].message.content
    messages.append({"role": "assistant", "content": reply})
    return messages

def stream_message(client: OpenAI, messages: list[dict]) -> list[dict]:
    stream = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=messages,
        stream=True,
    )
    full_reply = ""
    for chunk in stream:
        piece = chunk.choices[0].delta.content
        if piece:
            print(piece, end="", flush=True)
            full_reply += piece
    print()
    messages.append({"role": "assistant", "content": full_reply})
    return messages
```

```
# main.py
messages: list[dict] = []
while True:
    user_text = input("You: ")
    messages.append({"role": "user", "content": user_text})
    print("Assistant: ", end="", flush=True)
    messages = stream_message(client, messages)
```

Notice the order: the user message gets appended *before* the call. The assistant message gets appended *inside* `stream_message()`, *after* the streaming loop finishes. Get this order wrong and

either the model won't see the user's latest message, or the assistant's reply won't make it into history for the next turn.

Write and run the 3-message memory test from the README (message 3 refers back to message 1) before moving to Advanced.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Here's a first-pass trimming function, and where it plugs into the loop:

```
def estimate_tokens(messages: list[dict]) -> int:
    total_chars = sum(len(m["content"]) for m in messages)
    return total_chars // 4 # rough rule of thumb: ~4 characters per token

def trim_history(messages: list[dict], max_tokens: int = 3000) -> list[dict]:
    while estimate_tokens(messages) > max_tokens and len(messages) > 1:
        if messages[0]["role"] == "system":
            del messages[1] # keep the system prompt, drop the oldest real turn
        else:
            del messages[0]
    return messages
```

```
# main.py
messages: list[dict] = []
while True:
    user_text = input("You: ")
    messages.append({"role": "user", "content": user_text})
    messages = trim_history(messages)
    print("Assistant: ", end="", flush=True)
    messages = stream_message(client, messages)
```

Notice `trim_history()` runs *before* the call, not after — you want the list you're about to send to already be within budget, not the list you already sent. Try it with a low `max_tokens` (like `200`) and a long conversation, and watch it actually drop early messages instead of growing forever.

Difference between Basic, Intermediate, and Advanced: Basic and Intermediate build a working, growing memory with no upper bound. Advanced adds a real (if rough) budget check before every call, and a policy for what to do once you're over budget — drop the oldest non-system turns first — so a long-running conversation degrades gracefully instead of eventually failing outright.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Full solution: [Show me the solution](#)